Dimensionality Reduction: PCA and LDA

ML A-Z — Part 10

1 Why Reduce Dimensionality?

Datasets with many features (p large relative to the number of observations n) are hard to model and hard to visualize: models trained on high-dimensional data are prone to overfitting (the **curse of dimensionality**), and no plot can show more than 2–3 axes at once. **Dimensionality reduction** techniques replace the original p features with a smaller set of $k \ll p$ new features that retain as much of the useful structure in the data as possible.

This is **feature extraction**: unlike feature *selection* (which keeps a subset of the original features), extraction builds new features as combinations of the old ones. The two techniques covered here, PCA and LDA, both produce new features as linear combinations of the original ones, but they differ in what they optimize for:

- **PCA** is **unsupervised** — it ignores the target y entirely and looks only at the variance of X.
- **LDA** is **supervised** — it uses the class labels y to find the directions that best separate the classes.

Both are typically applied after feature scaling (standardization), since both rely on variance/covariance computations that are sensitive to the scale of each feature.

2 Principal Component Analysis (PCA)

2.1 Goal

PCA looks for the directions in feature space along which the data varies the most, under the reasoning that high-variance directions carry the most information, while low-variance directions are closer to noise. It projects the data onto the top k such directions, called **principal components**, discarding the rest.

2.2 Derivation: Maximizing Variance

Let $X \in \mathbb{R}^{n \times p}$ be the (mean-centered) data matrix. A direction is a unit vector $\mathbf{u} \in \mathbb{R}^p$, and projecting each observation onto it gives a new scalar feature $X\mathbf{u}$. Its variance is

$$\mathrm{Var}(X\mathbf{u}) = \frac{1}{n}\mathbf{u}^\top X^\top X\,\mathbf{u} = \mathbf{u}^\top \Sigma\,\mathbf{u},$$

where $\Sigma = \frac{1}{n}X^\top X$ is the $(p \times p)$ covariance matrix of the features. PCA's first component is the unit vector $\mathbf{u}_1$ that maximizes this quantity,

$$\mathbf{u}_1 = \arg\max_{\|\mathbf{u}\|=1} \mathbf{u}^\top \Sigma \mathbf{u}.$$

Since Σ is symmetric, it is diagonalizable with real eigenvalues: $\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i$, with eigenvectors $\mathbf{v}_i$ orthonormal and eigenvalues ordered $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_p \geq 0$. By the Rayleigh quotient argument, $\mathbf{u}^\top \Sigma \mathbf{u}$ is maximized (over unit vectors) exactly at $\mathbf{u} = \mathbf{v}_1$, giving maximum value λ_1. So:

- The **principal components** are the eigenvectors of Σ, ordered by eigenvalue.
- Each eigenvalue λ_i *is* the variance captured by its component: $\mathrm{Var}(X\mathbf{v}_i) = \lambda_i$.
- Components are mutually orthogonal ($\mathbf{v}_i^\top \mathbf{v}_j = 0$ for $i \neq j$), so they capture non-overlapping directions of variation — the second component maximizes remaining variance subject to being orthogonal to the first, and so on.

2.3 Algorithm

Algorithm 1 PCA

1: Standardize X (zero mean, unit variance per feature)
2: Compute the covariance matrix $\Sigma = \frac{1}{n} X^\top X$
3: Compute eigenvalues $\lambda_1 \geq \cdots \geq \lambda_p$ and eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_p$ of Σ
4: Choose k and form $V_k = [\mathbf{v}_1 \cdots \mathbf{v}_k]$
5: Project: $X_{\text{reduced}} = X V_k \in \mathbb{R}^{n \times k}$

2.4 Choosing k: Explained Variance

The fraction of total variance retained by the first k components is

$$\text{explained variance ratio} = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{p} \lambda_i},$$

since $\sum_i \lambda_i = \mathrm{tr}(\Sigma)$ is the total variance in the data. A common rule of thumb is to pick the smallest k that retains e.g. 95% of total variance, or to plot cumulative explained variance against k (a **scree plot**) and look for an elbow.

3 Linear Discriminant Analysis (LDA)

3.1 Goal

Where PCA asks "which directions have the most variance?", LDA asks "which directions best separate the known classes?" It is supervised: it uses the labels y to find projections that pull different classes' means apart while keeping each class's own spread tight.

3.2 Scatter Matrices

For C classes, let $\boldsymbol{\mu}_c$ be the mean of class c and $\boldsymbol{\mu}$ the overall mean. Define:

- **Within-class scatter** (spread inside each class, summed over classes):

$$S_W = \sum_{c=1}^{C} \sum_{i \in c} (\mathbf{x}_i - \boldsymbol{\mu}_c)(\mathbf{x}_i - \boldsymbol{\mu}_c)^\top$$

- **Between-class scatter** (spread of the class means around the overall mean, weighted by class size n_c):

$$S_B = \sum_{c=1}^{C} n_c (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^\top$$

3.3 Derivation: Maximizing Class Separability

LDA looks for a direction $\mathbf{u}$ that maximizes between-class scatter relative to within-class scatter after projection, i.e. it maximizes the **Fisher criterion**,

$$J(\mathbf{u}) = \frac{\mathbf{u}^\top S_B \mathbf{u}}{\mathbf{u}^\top S_W \mathbf{u}}.$$

Intuitively: the numerator is large when the projected class means are far apart, and the denominator is small when each projected class is tightly clustered around its own mean — exactly the two properties that make classes easy to separate. This is a generalized Rayleigh quotient, and its maximizers are the eigenvectors of $S_W^{-1} S_B$, ordered by eigenvalue:

$$S_W^{-1} S_B \, \mathbf{v}_i = \lambda_i \mathbf{v}_i.$$

These eigenvectors are the **linear discriminants**. Since S_B is a sum of C rank-one-ish terms centered at a common overall mean, it has rank at most $C - 1$, so there are at most $C - 1$ linear discriminants with nonzero eigenvalue — unlike PCA, LDA's number of usable components is capped by the number of classes.

3.4 Algorithm

Algorithm 2 LDA

1: Standardize X
2: Compute per-class means $\boldsymbol{\mu}_c$ and overall mean $\boldsymbol{\mu}$
3: Compute within-class scatter S_W and between-class scatter S_B
4: Compute eigenvalues/eigenvectors of $S_W^{-1} S_B$
5: Choose $k \leq C - 1$ and form V_k from the top-k eigenvectors
6: Project: $X_{\text{reduced}} = X V_k$

4 PCA vs. LDA

	PCA	LDA
Uses labels y?	No (unsupervised)	Yes (supervised)
Optimizes	Variance of projected data	Between/within class scatter ratio
Matrix diagonalized	Σ (covariance)	$S_W^{-1} S_B$
Max useful components	p (# features)	$C - 1$ (# classes -1)
Typical use	Compression, visualization, noise reduction	Preprocessing for classification

In the accompanying notebooks, both are applied to the Wine dataset (13 features, 3 cultivar classes), reduced to 2 components, and fed into a logistic regression classifier. LDA reaches perfect test accuracy there precisely because it optimizes directly for class separability rather than for variance, which incidentally may or may not align with what separates the classes.

5 Key Takeaways

- Both PCA and LDA are linear feature-extraction methods: the new features are linear combinations of the old ones, found via an eigendecomposition.
- PCA is unsupervised and maximizes variance; it can find up to p components and is a good default when there is no label, or when the goal is compression/visualization rather than classification.
- LDA is supervised and maximizes class separability; it is capped at $C - 1$ components and tends to work better as a preprocessing step specifically for classification, since it optimizes for the thing that actually matters for that task.
- Kernel PCA extends PCA to nonlinear structure by implicitly mapping data into a higher-dimensional space (via the kernel trick) before finding principal components — useful when classes are not linearly separable in the original feature space.